

Projet Motus – Rapport de projet

Master 2 MIAAGE SITN Apprentissage 2025-2026 — Projet Applications Web orientées Services (M. Menceur)

Binôme : Fahd Derradji & Guillaume Karaouane

Lien GitHub : <https://github.com/GuillaumeK75015/Projet-Motus-Microservices>

1 Compilation / exécution du projet

Prérequis

- Java 26 + Maven (ou `./mvnw` fourni dans chaque module)
- Docker Desktop (méthode recommandée) **ou** PostgreSQL 16 en local **ou** MiniKube

Méthode recommandée — Docker Compose

```
docker-compose up --build
```

Cette commande démarre PostgreSQL, les 4 microservices métier, l'**API Gateway** ainsi que **Prometheus** et **Grafana**, initialise les 3 bases de données via `init-db.sql`, puis expose l'application sur `http://localhost:8080` (point d'entrée unique). Prometheus est disponible sur `:9090`, Grafana sur `:3000` (identifiants `admin/admin`).

Sans Docker

```
CREATE DATABASE db_players;
CREATE DATABASE db_history;
CREATE DATABASE db_motus;
```

```
cd player-service      && ./mvnw spring-boot:run
cd dictionary-service  && ./mvnw spring-boot:run
cd history-stat-service && ./mvnw spring-boot:run
cd motus-game-service  && ./mvnw spring-boot:run
cd api-gateway         && ./mvnw spring-boot:run
```

Puis ouvrir `http://localhost:8080`.

Avec MiniKube (déploiement Kubernetes)

```
minikube start --cpus=4 --memory=6144
& minikube -p minikube docker-env --shell powershell | Invoke-Expression
docker build -t player-service:latest ./player-service
docker build -t dictionary-service:latest ./dictionary-service
docker build -t history-stat-service:latest ./history-stat-service
docker build -t motus-game-service:latest ./motus-game-service
docker build -t api-gateway:latest ./api-gateway
kubectl apply -f k8s/
kubectl wait --for=condition=Ready pod --all --timeout=120s
minikube service api-gateway --url
```

Les manifests (`k8s/`) définissent un **Deployment** + **Service** par microservice, un **Secret** pour les identifiants PostgreSQL, et des **readinessProbe**/**livenessProbe** HTTP basées sur **Actuator** (`/actuator/health/readiness`, `/actuator/health/liveness`). Ils ajoutent aussi des **resources.requests/limits** et **JAVA_TOOL_OPTIONS** par service (voir §2.7) pour un démarrage stable sous MiniKube.

Comptes de test

- **Joueur** : inscription libre (pseudo + email + mot de passe), ou bouton « *Jouer en invité* » pour une partie immédiate sans compte.
- **Administrateur** : compte créé automatiquement au démarrage de **player-service** (admin / Admin1234!, configurable dans `application.properties`).

2 Documentation technique

2.1 Architecture — découpage en microservices

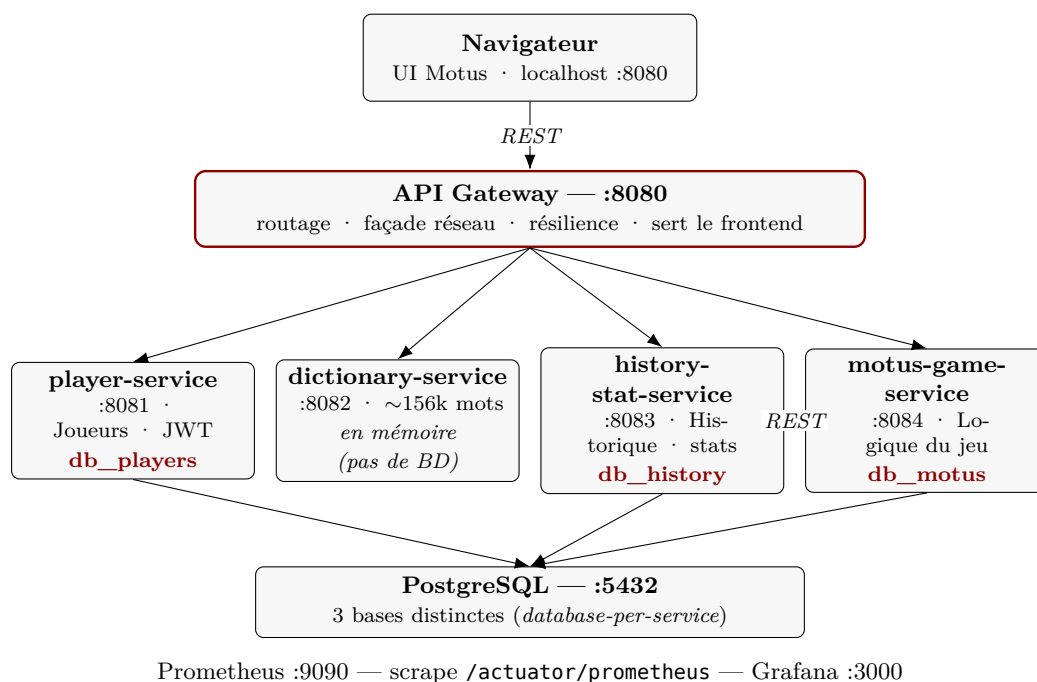


Figure 1 — Architecture microservices : le navigateur ne dialogue qu'avec l'API Gateway (point d'entrée unique), qui route vers les quatre services métier ; chaque service possède sa propre base (*database-per-service*).

Chaque service possède sa **propre base de données** (pattern *database-per-service*) et n'accède jamais directement à celle d'un autre service : toute communication passe par API REST (RestTemplate). Le navigateur ne parle qu'à l'**API Gateway** (figure 1), qui route vers **motus-game-service** pour le jeu et vers **player-service/history-stat-service** pour les comptes et l'historique ; **motus-game-service** reste purement métier et ne fait plus office de proxy.

2.2 Diagramme de classes « métier » (vision conceptuelle)

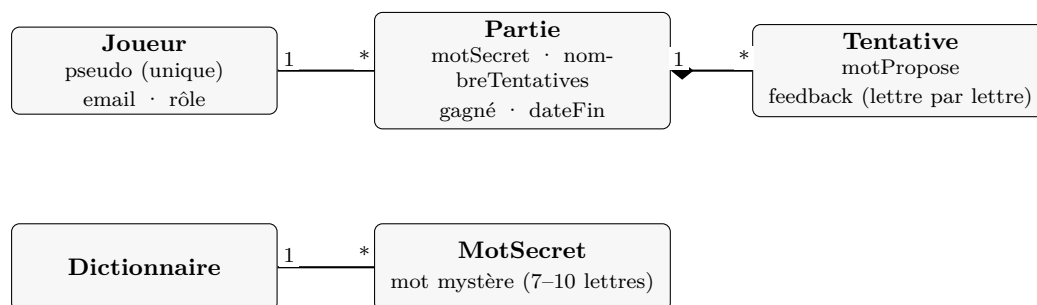


Figure 2 — Diagramme de classes conceptuel. Association **Joueur** 1---***Partie** ; composition **Partie** ◇---***Tentative** (une tentative n'existe pas sans sa partie).

2.3 Diagrammes de classes — entités JPA par service

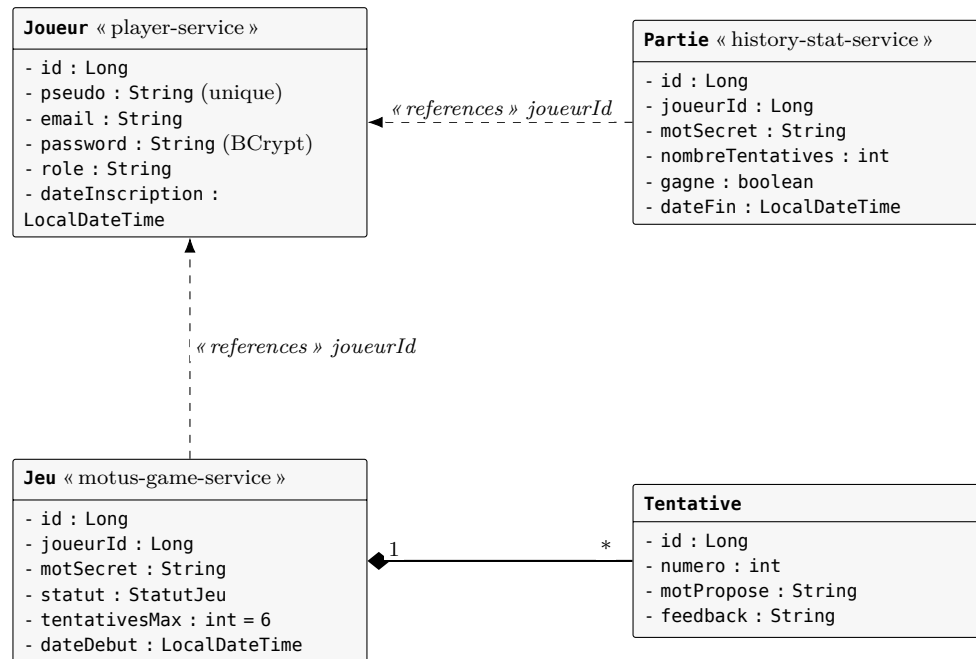


Figure 3 — Entités JPA réparties par microservice (visibilité - = privé). Composition $\text{Jeu} \diamond \text{Tentative}$ (cascade ALL) ; les flèches en pointillés sont des références *logiques* inter-services (*joueurId*), et non des associations JPA réelles.

dictionary-service : pas d'entité persistante — dictionnaire chargé en mémoire au démarrage (environ 156 000 mots valides de 7 à 10 lettres, filtrés depuis un lexique brut de 336 000 formes).

2.4 API REST (extrait des endpoints principaux)

Tous les endpoints ci-dessous sont exposés côté client via l'**API Gateway** (<http://localhost:8080>), point d'entrée unique de l'application.

Méthode	Endpoint (via l'API Gateway /api/proxy/...)	Accès
POST	/api/proxy/players — inscription (mot de passe requis)	public
POST	/api/proxy/players/guest — partie en invité, sans compte	public
POST	/api/proxy/auth/login — connexion (joueur ou admin) → JWT	public
GET	/api/proxy/players — liste des joueurs	ADMIN
DELETE	/api/proxy/players/{id} — suppression (cascade historique)	ADMIN
POST	/api/games/start, /api/games/{id}/guess — jouer	applicatif (joueurId)
GET	/api/proxy/stats/{id}, /api/proxy/classement — stats / classement	public
GET	/api/proxy/search?joueurId=&date=&gagne= — recherche des parties	ADMIN

2.5 Choix techniques

- **Spring Boot 4.1.0 / Java 26**, spring-boot-starter-webmvc + Spring Data JPA.
- **PostgreSQL** : une base dédiée par microservice (db_players, db_history, db_motus), dictionary-service reste sans base (dictionnaire en mémoire).

- **Spring Security + JWT** (ajouté en cours de projet, cf. bilan) :
 - Authentification **stateless** par jeton JWT (HS384), secret partagé entre **player-service** et **history-stat-service**.
 - Mots de passe hashés en **BCrypt**; aucun mot de passe n'est jamais renvoyé par l'API (@JsonProperty(access = WRITE_ONLY)).
 - Deux rôles : **ROLE_USER** (joueur) et **ROLE_ADMIN** (accès à la liste/recherche des parties, gestion des joueurs).
 - **Mode invité** : partie jouable sans inscription (compte éphémère sans mot de passe).
 - Protection anti-élévation de privilège : le rôle est forcé à **ROLE_USER** côté serveur à l'inscription, quel que soit le contenu envoyé par le client.
- **Conteneurisation** : un **Dockerfile** multi-stage (build Maven puis image d'exécution) par service, orchestré par **docker-compose.yml**.
- **Déploiement Kubernetes** : manifests **Deployment + Service** par microservice dans **k8s/**, **Secret** pour les identifiants PostgreSQL, **readinessProbe/livenessProbe** HTTP basées sur Spring Boot Actuator.
- **Frontend** : une unique page HTML/CSS/JS (sans framework), servie statiquement par **api-gateway**, qui route aussi les appels API vers les services internes pour éviter le CORS.

API Gateway (api-gateway, service 5) : point d'entrée unique de l'application (:8080), extrait de l'ancien ProxyController de **motus-game-service** pour se conformer au pattern *API Gateway* du cours. Il centralise :

- le relai des appels vers **player-service/history-stat-service** (auth, joueurs, stats, classement, recherche admin), avec transfert du header **Authorization**, erreurs HTTP 4xx/5xx renvoyées telles quelles, panne réseau → 503;
- le routage générique **/api/games/**** vers **motus-game-service**, qui reste désormais purement métier;
- le service du frontend statique (**index.html**).

Résilience (Resilience4j) : les appels inter-services par **RestTemplate** sont enveloppés dans des clients dédiés (**PlayerClient**, **HistoryClient**, **DictionaryClient**, **DownstreamClient**) annotés **@CircuitBreaker/@Retry**. Seules les pannes réseau déclenchent retry puis ouverture du circuit — une erreur métier 4xx/5xx du service appelé est traitée normalement. Objectif : éviter qu'une panne d'un service ne fasse échouer en cascade toute une partie en cours.

Observabilité (Actuator + Prometheus + Grafana) : chaque service expose **spring-boot-starter-actuator**, avec des groupes de santé **readiness/liveness** dédiés aux probes Kubernetes. Prometheus scrape les 5 services toutes les 15s et Grafana permet de visualiser latence, taux d'erreur et JVM/GC en direct.

HATEOAS (niveau 3 de Richardson) : **spring-boot-starter-hateoas** ajouté à **player-service** et **motus-game-service**. Les liens sont ajoutés sous une clé **_links** séparée, sans impact sur le frontend actuel.

CI/CD (GitHub Actions) : **.github/workflows/ci.yml** compile et teste (**mvn verify**) chacun des 5 modules en matrice à chaque push/PR sur **main**, puis valide que les 5 images Docker se construisent.

Tests : JUnit 5 + Mockito pour les contrôleurs et services, tests d'intégration Spring Boot sur **player-service** et **api-gateway**, test d'intégration base **@DataJpaTest** (H2) sur **history-stat-service**.

2.6 Robustesse, correction & expérience de jeu

Codes HTTP sémantiques (motus-game-service) : une petite hiérarchie d'exceptions (`NotFoundException` → 404, `InvalidGuessException` → 400, `ServiceUnavailableException` → 503) remplace le `RuntimeException` générique qui retombait systématiquement en 400.

Correctif — cache du dictionnaire (fiabilité + dégradation gracieuse) : le `DictionaryClient` conservait un garde `cache.isEmpty()` pour décider de recharger le dictionnaire ; or `ajouterAuCache(motSecret)` insère le mot secret à chaque début de partie, si bien qu'après un échec de chargement initial (course au démarrage) le cache n'était plus jamais rechargé et **tous les mots sauf le secret étaient refusés**. Remplacé par un indicateur explicite `chargeComplet` + un **repli autoritaire** vers `/api/dictionary/exists/{mot}` si le cache reste incomplet.

Performance — filtrage & agrégation en base (history-stat-service) : la recherche multi-critères et le classement chargeaient toutes les parties (`findAll()`) puis filtraient/groupeaient en mémoire. Ils utilisent désormais des requêtes JPQL dédiées (`search(...)`, `aggregateClassement()`).

Partage du score (façon Wordle) : en fin de partie, un bouton « Partager mon score » génère une grille d'emojis à partir de l'état de partie, copiée dans le presse-papier.

Correctif — recherche multi-critères sur PostgreSQL réel : la requête JPQL `search(...)` passait le test `@DataJpaTest` (H2) mais **échouait à 100 % des appels en base réelle** (`PSQLException: could not determine data type of parameter, SQLState 42P18`) : PostgreSQL ne peut pas déterminer le type d'un paramètre qui n'apparaît que dans un `? IS NULL` sans autre contexte typé, contrairement à H2 qui est plus permissif. Corrigé en castant explicitement chaque paramètre optionnel **uniquement** dans son test `IS NULL` (`CAST(:gagne AS boolean) IS NULL OR p.gagne = :gagne`) ; caster aussi le côté comparaison casse l'inférence de type d'Hibernate pour les paramètres non numériques (`cannot cast type bytea to boolean` pour `:gagne`). Revérifié avec un vrai `docker compose up` + PostgreSQL sur les 6 combinaisons de paramètres : toutes renvoient désormais **200** avec les données correctes.

2.7 Déploiement Kubernetes — stabilité sous MiniKube

Constat : sous MiniKube (driver Docker), les 5 services + PostgreSQL redémarrèrent en boucle (2 à 3 restarts par pod avant stabilisation, 5 à 7 minutes de démarrage) alors que les mêmes images tournaient sans souci en Docker Compose.

Cause : `kubectl describe node` annonce 8 CPU / 7,6 Gi de RAM (valeurs lues côté hôte), mais le conteneur Docker du nœud MiniKube est en réalité plafonné en dur par cgroup à 2 CPU / 4 Go. Aucun des 5 `Deployment` ne déclarait de `resources.requests/limits` ni de réglage de heap JVM (contrairement à `docker-compose.yml`, qui définit déjà `JAVA_TOOL_OPTIONS/mem_limit`) : les 6 JVM se disputaient un budget CPU réel bien inférieur à ce que Kubernetes croyait disponible, le démarrage de Spring Boot dépassait alors les seuils des probes, et le kubelet tuait puis relançait les pods en boucle.

Correctifs :

- Cluster relancé avec des ressources réelles alignées sur ce que Kubernetes annonce (`minikube start --cpus=4 --memory=6144`).
- Ajout de `resources.requests/limits` + `JAVA_TOOL_OPTIONS: -XX:MaxRAMPercentage=65 -Xss512k` dans les 5 manifests (`k8s/*.yaml`), en reprenant les valeurs mémoire déjà définies dans `docker-compose.yml`.
- Cache Maven partagé entre builds (`RUN --mount=type=cache,id=m2-repo,target=/root/.m2`) dans les 5 `Dockerfile`, pour ne pas retélécharger tout l'arbre de dépendances à chaque `docker`

build (temps de build divisé par 2 à 10 selon le service — sans impact sur l'indépendance runtime de chaque microservice, qui reste un JAR autonome déployable seul).

Résultat : les 6 pods atteignent **1/1 Running** en ~80 secondes sans aucun restart (contre 5-7 minutes et 2-3 restarts/pod auparavant).

3 Bilan du projet

Ce que nous avons aimé — Découper une application en microservices indépendants, chacun avec sa base de données et sa responsabilité propre, et les faire collaborer par API REST. Voir concrètement l'intérêt du pattern *proxy/façade* pour donner au frontend un point d'entrée unique sans exposer ni CORSer les autres services.

Ce que nous avons appris — La mise en place de l'authentification JWT stateless partagée entre plusieurs services ; les pièges classiques de Jackson (`@JsonIgnore` bloque aussi la désérialisation, pas seulement la sérialisation) ; l'importance de `@Transactional` pour les méthodes de suppression dérivées de Spring Data JPA ; la configuration Kubernetes (probes, secrets, **Deployment/Service**) et son couplage avec les règles de sécurité applicative ; l'importance de toujours échapper les données utilisateur affichées côté client pour éviter les failles XSS.

Ce que nous avons moins aimé — Le dictionnaire de mots fourni (`mots.txt`) est un lexique brut de formes fléchies, pas une liste de mots courants : on y trouve beaucoup de conjugaisons rares, ce qui rend le jeu localement plus difficile qu'un Motus « grand public ».

Réussites — Architecture microservices fonctionnelle de bout en bout (inscription, partie, historisation, statistiques, classement, administration) ; sécurisation ajoutée sans casser le parcours joueur existant ; suppression en cascade d'un compte et de son historique à travers deux services indépendants ; déploiement testé en local, en Docker Compose et en manifests Kubernetes. Après relecture du cours, quatre écarts identifiés ont été comblés sans régression (33 tests toujours au vert sur **motus-game-service**, 11 sur **player-service**) : extraction d'une **API Gateway** dédiée, **observabilité** (Actuator + Prometheus/Grafana), **résilience** (Resilience4j) et une première **pipeline CI** (GitHub Actions).

Difficultés — Plusieurs bugs subtils découverts en testant réellement chaque fonctionnalité plutôt qu'en se fiant à la seule compilation : une dépendance Jackson manquante en scope *compile* faisait planter **motus-game-service** au démarrage ; le renommage de certains packages d'auto-configuration Spring Security entre versions de Spring Boot ; une image Docker reconstruite mais pas redéployée (cache) qui masquait un correctif déjà appliqué ; une faille XSS stockée détectée en revue de code puis corrigée ; une requête JPQL à paramètres optionnels validée par un test `@DataJpaTest` (H2) mais cassée à 100 % sur PostgreSQL réel — un rappel que « les tests passent » ne remplace pas un run réel contre l'infrastructure cible ; et, côté MiniKube, un nœud dont le vrai plafond cgroup (CPU/RAM) ne correspondait pas à ce que `kubectrl describe node` annonçait, provoquant des redémarrages en boucle des pods faute de `resources.requests/limits` déclarées.